Input Specification

An input is one mapping from key names to values. This section lists every key, in groups, with its shape, its default and the cells that read it. Section 3 gives the file formats and the keys that each cell type needs.

Cells and abbreviations. A *cell* is a pair (i, ℓ) of a train and a component. Each cell has its own degradation model, repair law, reliability bound, linear form and encoding. The column *Applies to* names the cells that read a key. It uses these abbreviations: G (Gaussian), IG (inverse Gaussian), W (Wiener), Ga (Gamma), RF (rainflow). For a rainflow cell it names the tail bound: Mk (Markov), Ca (Cantelli), Ho (Hoeffding), Be (Bernstein), Ch (Chernoff). The word *all* means every cell.

Shapes. A *cell value* is a scalar, a list of L numbers, or an $F \times L$ array. A scalar is copied to every cell. A list of L numbers holds one value per component and is copied to every train. A *selector* has the same three shapes, with strings in place of numbers. A *profile* holds one value per cell, mission and day; its shapes are given in Section 1.7. A *flag* is `true` or `false`. No other broadcasting is accepted.

Absent entries. At a cell where a key does not apply, the entry of an array may be `null`, and the key may be omitted. At a cell where a key applies, `null` is an error, also when the key has a default. A broadcast value (a scalar, a list, or a profile without the axis F) is read only where the key applies; it gives no warning. A non-`null` entry of an array that has one entry per cell (an $F \times L$ array, or a profile with the axes F and L), at a cell where the key does not apply, is ignored with a `UserWarning`.

Keys. The names in the tables are the only keys that an input may hold. Names are case sensitive. Any other key raises an error (Section 1.11).

Sizes and Horizon

Key	Shape	Default	Applies to	Meaning
`F`	integer	required	all	Number of trains. Must exceed M
`M`	integer	required	all	Number of missions. Every mission is flown by one train on every day
`L`	integer	inferred	all	Number of components per train. When `model` is a list or an array, L is the length of its component axis, and a given `L` must equal it. Required when `model` is one string and $L > 1$
`H`	integer or `[H1, H2]`	required	all	Horizon (below). A scalar H means $H_1 = H_2 = H$

The horizon has $T = H_1 + H_2$ days. Days $1, \ldots, H_1$ form the *transitory phase*. Days $H_1+1, \ldots, T$ form the *operating phase*. Both H_1 and H_2 are positive integers. The operating phase is meant to repeat without end; the loop constraint of Section **??** makes every repetition safe. The profiles of Section 1.7 have H_2 entries for the operating phase and, optionally, H_1 entries for the transitory phase. Index $k = 0$ holds the initial state.

Selectors

Key	Shape	Default	Applies to	Meaning
`model`	selector	required	all	Degradation model of the cell: `gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`
`repair_model`	selector	`ardinf`	all	Repair law of the cell (Section **??**): `ardinf` (the repair reduces the whole level) or `ard1` (it reduces the level accumulated since the last intervention). Every model accepts both
`bound_method`	selector	`cantelli`	RF	Tail bound of the reliability row (Section **??**): `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`
`reliability_impl`	selector	`exact`	G, W, RF (Ca, Be, Ho)	Form of the reliability row (Section **??**): `exact`, `tangent` or `pwl`. A cell whose row is linear (IG, Ga, Mk, Ch) is built in the form `exact`, and the value is ignored
`encoding`	selector	`bigm`	all	Encoding of the dynamics rows (Section **??**): `bigm` or `indicator`
`component_names`	list of L strings	`Component` n for component n	all	Name of each component, shown by the plots. A name is not empty after blanks are removed

Reliability

Key	Shape	Default	Applies to	Meaning
`tau`	cell value	required	all	Degradation threshold $\tau_{i\ell} > 0$
`epsilon`	cell value	required	all	Reliability level $\varepsilon_{i\ell} \in (0, 1)$: the probability that the degradation exceeds $\tau_{i\ell}$ must not exceed it on any day. At G and W cells $\varepsilon_{i\ell} < 0.5$
`tangent_ref`	cell value	0.5	cells with form `tangent`	Relative tangent point in $[0, 1)$: the tangent is taken at the mean $\mu_r = \texttt{tangent_ref} \cdot \tau$
`pwl_points`	integer	8	cells with form `pwl`	Number of segments of the piecewise linear form

Remark (range of ε). The Gaussian and Wiener rows contain $\big(\Phi^{-1}(1-\varepsilon)\big)^2$. This number is positive only for $\varepsilon < 0.5$. The variance range $V_{\max} = \tau^2/\big(\Phi^{-1}(1-\varepsilon)\big)^2$ is the largest variance that the row admits. It is the natural range of the variance, and the big-M constant of the variance rows (Section **??**). The ratio $V_{\max}/\tau^2$ is 0.18 at $\varepsilon = 0.01$, 0.61 at 0.1, 3.6 at 0.3 and about 1600 at 0.49. The constant is therefore moderate unless ε is close to 0.5. The other models

and bounds have no limit below $\varepsilon = 1$: for example, the variance cap $\tau^2\varepsilon/(1-\varepsilon)$ of the Cantelli row is finite for every $\varepsilon < 1$.

Repair and Starting Anchors

Key	Shape	Default	Applies to	Meaning
`rho`	cell value	required	all	Repair efficiency $\rho_{i\ell} \in (0, 1]$
`anchor_mu_0`	cell value	μ_0	`ard1` cells	Starting anchor $g^{\mu}_{i\ell 0}$ of the mean
`anchor_v_0`	cell value	v_0	`ard1` cells at G, RF (Ca, Be)	Starting anchor $g^{v}_{i\ell 0}$ of the variance
`anchor_R_0`	cell value	R_0	`ard1` cells at RF (Ho)	Starting anchor $g^{R}_{i\ell 0}$ of the Hoeffding budget
`anchor_K_0`	cell value	K_0	`ard1` cells at RF (Ch)	Starting anchor $g^{K}_{i\ell 0}$ of the Chernoff quantity

Under the law `ard1`, a repair removes the fraction ρ of the level accumulated since the last intervention (Section **??**). Before the first intervention, the reference level is the starting anchor of the state. The default of each starting anchor is the initial value of the same state: μ_0, v_0, R_0 and K_0 (Section 1.5). The largest allowed value is this default, because damage only grows between interventions (Section 1.12). A larger anchor gives a weaker repair and a higher state, so the default never overstates what a repair achieves. For example, let $\mu_0 = 0.5$ and $\rho = 0.5$, and let a repair take place on day 1 before any mission. With the starting anchor 0 the mean falls to 0.25; with the default anchor 0.5 it stays at 0.5. The input is exact when the maintenance history of the component is known.

Initial State

Key	Shape	Default	Applies to	Meaning
`mu_0`	cell value	required	all	Initial mean degradation $\mu_{0,i\ell}$, with $0 \le \mu_{0,i\ell} < \tau_{i\ell}$
`v_0`	cell value	0	G, W, RF (Ca, Be)	Initial variance $v_{0,i\ell} \ge 0$. The value 0 means a known initial damage
`support_0`	cell value	0	RF (Be)	Width $b^0_{i\ell}$ of the range of the initial damage. The value 0 means a known initial damage
`R_0`	cell value	0	RF (Ho)	Initial value $R_{0,i\ell}$ of the Hoeffding budget: the square of the width of the range of the initial damage
`cgf_0`	cell value	$s\,\mu_0$	RF (Ch)	Initial value $K_{0,i\ell}$ of the Chernoff quantity: the cumulant generating function of the initial damage at $s_{i\ell} =$ `s_chernoff`. It is at least $s\,\mu_0$

The defaults describe a known initial damage: the mean μ_0 with no spread, so that $v_0 = b^0 = R_0 = 0$ and $K_0 = s\,\mu_0$. The optional keys describe a random initial damage. Section **??** states why these values keep the certificate valid.

Replacement

Key	Shape	Default	Applies to	Meaning
`replacement_mu`	cell value	0	all	Mean degradation $\mu_{i\ell}^{\text{new}}$ after a replacement
`replacement_v`	cell value	0	G, W, RF (Ca, Be)	Variance $v_{i\ell}^{\text{new}}$ after a replacement
`support_new`	cell value	0	RF (Be)	Width $b_{i\ell}^{\text{new}}$ of the range of the damage after a replacement
`R_new`	cell value	0	RF (Ho)	Hoeffding budget $R_{i\ell}^{\text{new}}$ after a replacement
`cgf_new`	cell value	$s\,\mu^{\text{new}}$	RF (Ch)	Chernoff quantity $K_{i\ell}^{\text{new}}$ after a replacement; at least $s\,\mu^{\text{new}}$
`replacement_as_new`	flag	`false`	all	If `true`, a replacement gives a component in the zero state, and every replacement input of this table must be 0
`allow_replacement`	flag	`true`	all	If `false`, no replacement is possible: the variables x^r and their rows are not built

The state after a replacement comes from these keys for every model. The defaults give a part in the zero state. Section **??** states the replacement rows; Section **??** states the reset of R and K. For Gamma, the rate of the state after a replacement is `gamma_beta_new` (Section 1.8).

Increment Profiles and the Phase Lookup

A *profile* gives the parameters of the damage increment when train i flies mission j on a day, for component ℓ. Its canonical shape is (F, L, M, H_*): train, component, mission, day of the phase. The last axis has H_2 entries for an operating profile and H_1 entries for a transitory profile. The parser accepts these shapes and expands each of them to (F, L, M, H_*) before it builds any row (*eager broadcasting*):

$$\text{scalar}, \quad (M), \quad (L, M), \quad (L, M, H_*), \quad (F, L, M), \quad (F, L, M, H_*).$$

A missing axis is repeated along that axis. A three-dimensional array whose shape fits both (L, M, H_*) and (F, L, M) is read as (L, M, H_*); this happens only when $F = L = M = H_*$. The parser cannot detect an array whose mission axis and component axis are swapped when $M = L$. Missions have index $j \in \{1, \dots, M\}$ in a profile; a depot day or an idle day adds no increment.

Phase lookup. Let p be an operating profile and p^{tr} the transitory profile of the same key, if it is given. After the phase lookup, the value used on day $k \in \{1, \dots, T\}$ is

$$p_{i\ell jk}^{\text{inc}} = \begin{cases} p_{i\ell jk}^{\text{tr}} & k \le H_1 \text{ and } p^{\text{tr}} \text{ is given}, \\ p_{i\ell j,\,((k-1) \bmod H_2)+1} & k \le H_1 \text{ and } p^{\text{tr}} \text{ is not given}, \\ p_{i\ell j,\,((k-H_1-1) \bmod H_2)+1} & k > H_1. \end{cases} \tag{1}$$

Every profile key is looked up on its own. A transitory profile may be given for some keys of a cell and not for others. For a scalar H without transitory profile, both phases read the same H entries, one after the other. For $H = [2, 3]$ without transitory profile, days $1, \dots, 5$ read the operating entries $1, 2, 1, 2, 3$. With a transitory profile, days $1, 2$ read its entries $1, 2$ and days $3, 4, 5$ read the operating entries $1, 2, 3$. The symbols $\mu_{i\ell jk}^{\text{inc}}$, $v_{i\ell jk}^{\text{inc}}$, $b_{i\ell jk}^{\text{inc}}$ and $K_{i\ell jk}^{\text{inc}}$ of the formulation denote the values after this lookup.

Key	Shape	Default	Applies to	Meaning
`mu`	profile	required	all	Mean $\mu^{\text{inc}}_{i\ell jk} \geq 0$ of the increment. For Ga the increment has shape $\mu\,\beta$ and rate β, so that its mean is μ
`v`	profile	required	G, RF (Ca, Be)	Variance $v^{\text{inc}}_{i\ell jk} \geq 0$ of the increment
`support`	profile	required	RF (Ho, Be)	Width $b^{\text{inc}}_{i\ell jk} \geq 0$ of the range of the increment: the increment lies in an interval of this length
`cgf`	profile	required	RF (Ch)	Cumulant generating function $K^{\text{inc}}_{i\ell jk} = \ln \mathbb{E}\, e^{sX}$ of the increment X at $s = s_{i\ell}$
`s_chernoff`	cell value	required	RF (Ch)	Chernoff parameter $s_{i\ell}$, with $s_{i\ell} > \ln(1/\varepsilon_{i\ell})/\tau_{i\ell}$
`gamma_beta`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_2)	required	Ga	Rate $\beta_{i\ell jk} > 0$ of the Gamma increment. A shape without a day axis is repeated along the days
`mu_trans`, `v_trans`, `support_trans`, `cgf_trans`	transitory profile	none	as the key without the suffix	Increment parameters of the transitory phase. A transitory profile requires its operating profile
`gamma_beta_trans`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_1)	none	Ga	Rates of the transitory phase

The key `support` keeps one meaning in every bound: the width of the range of the increment. A width of 0 means that the increment is known.

Model Constants

Key	Shape	Default	Applies to	Meaning
`sigma`	cell value	required	W	Diffusion constant $\sigma_{i\ell} > 0$: every mission adds $\sigma^2_{i\ell}$ to the variance
`eta`	cell value	required	IG	Ratio $\eta_{i\ell} = \mu/\lambda > 0$ of the mean to the shape parameter of the inverse Gaussian law of the state (Section **??**)
`v_max_user`	cell value	none	G, W, RF (Ca, Be)	Upper bound $V^{\text{user}}_{\max,i\ell} > 0$ on the variance. If it is given, the row $v_{i\ell k} \leq V^{\text{user}}_{\max,i\ell}$ holds on every day. If it is not given, only the reliability row bounds the variance
`gamma_beta_0`	cell value	none (known damage)	Ga	Rate of the initial damage, which is a Gamma variable with mean μ_0. If it is omitted, the initial damage is the known constant μ_0, and it does not enter the smallest rate

Key	Shape	Default	Applies to	Meaning
`gamma_beta_new`	cell value	none (known damage)	Ga	Rate of the damage after a replacement, which is a Gamma variable with mean μ^{new}. If it is omitted, the damage after a replacement is the known constant μ^{new}, and it does not enter the smallest rate

For Gamma, the formulation computes the smallest rate of each cell from the rates of the terms with a positive mean: the entries of `gamma_beta` and `gamma_beta_trans`, and the rates `gamma_beta_0` and `gamma_beta_new` where they are given (Section **??**). No input gives this rate.

Costs and Objective

Key	Shape	Default	Applies to	Meaning
`C_M`	scalar	required	all	Cost $C_M \geq 0$ of one depot day of one train, whatever the number of jobs done on that day
`C_R`	cell value	required	all	Repair cost $C_{R,i\ell} \geq 0$ per unit of degradation removed. At a cell with $C_{R,i\ell} = 0$ the variable z and its rows are not built
`C_rep`	cell value	required	all	Replacement cost $C_{\text{rep},i\ell} \geq 0$
`C_D`	scalar	required	all	Damage regularisation coefficient $C_D \geq 0$. If $C_D = 0$, the variable u and its rows are not built, and `penalty_type` is ignored
`penalty_type`	string	`inf_norm`	all	Form of the damage regularisation (Section **??**): `inf_norm` or `quadratic`. The value `quadratic` requires the form `exact` at every cell

All costs are finite. The objective and its three modes (`objective_mode`, Section 1.10) are stated in Section **??**.

Run Options

Run options are keys of the input. They do not change the mathematical problem, except `objective_mode` and `evaluation_horizon`, which choose the objective.

Key	Shape	Default	Meaning
`verbose`	integer	1	Gurobi output flag: 0 for no log, 1 for the log

Key	Shape	Default	Meaning
`mip_gap`	number ≥ 0	0.1	Relative gap between the best solution and the best bound at which Gurobi stops (the MIP gap; MIP means mixed-integer program). With the default, the status `optimal` means that the best solution is at most 10 % above the best bound. The gap reached is reported as `mip_gap` in the result
`time_limit`	number > 0	3600	Time limit of Gurobi, in seconds
`fast`	flag	`false`	If `true`, a preset of Gurobi parameters that favours good solutions found early: `MIPFocus` $= 1$, `Heuristics` $= 0.5$, `ImproveStartGap` $= 0.5$ and `NoRelHeurTime` $= \max(2, 0.15\,$`time_limit`$)$
`gurobi_params`	mapping	none	Gurobi parameters, by name. They are set after every other option, and so override it
`objective_mode`	string	`total`	`total`: the cost of all T days; `operating_average`: the cost per day of the operating phase; `evaluation_total`: the projected cost over `evaluation_horizon` days. Each mode adds the damage penalty $C_D\,u$ once (Section **??**)
`evaluation_horizon`	integer	none	Evaluation horizon E. Required, and used, only if `objective_mode` is `evaluation_total`; $E > H_1$. If it is given in another mode, the result reports the projected cost for it (Section 2)
`progress_interval_seconds`	number > 0	5.0	Minimum time in seconds between two records of the progress report (Section 2). A record is also made whenever the number of solutions found changes
`relaxation_warm_start`	flag	`false`	If `true`, and no other warm start is applied, a relaxation start is computed before the solve (Section **??**)
`relaxation_time_limit`	number > 0	60.0	Time limit in seconds of the relaxation start

No input key selects a warm start from a previous result; Section **??** defines it. A horizon sweep is not an input option; the sweep functions of Section **??** read the same input and solve it for several horizons.

Rejected Keys

Each key of the table below names a quantity that has another name, or another meaning, in this specification. A key of the table raises an error. The message names the canonical key and, where the meaning differs, states the meaning of the canonical key. Any other key that is not in the tables of this section raises the error "unknown key", with the list of the canonical keys. The check on keys runs before any array is read.

Rejected key	Message: canonical key and meaning
`maintenance_type`	Use `repair_model`. Its values are `ardinf` (the repair reduces the whole level) and `ard1` (it reduces the level accumulated since the last intervention). In a file written with `maintenance_type`, the value `ARD1` corresponds to `ardinf`, and the value `ARA1` to `ard1`
`tail_bound`, `method`	Use `bound_method`; its values are `markov`, `cantelli`, `hoeffding`, `bernstein` and `chernoff`
`formulation`	Use `reliability_impl`. The value `exact` keeps its meaning. The value `lp` corresponds to `tangent` with `tangent_ref` equal to 0
`beta`	Use `gamma_beta`. It is a rate, and `beta` is a scale: `gamma_beta` equals 1/`beta`
`alpha_inc`	The Gamma increments are given by the mean `mu` and the rate `gamma_beta`. The shape of an increment is `mu` times `gamma_beta`
`gamma_beta_bound`	Remove it. The formulation computes the smallest rate of each cell from the rates of the cell (Section **??**)
`gamma_calibration_method`	Remove it. The formulation has no calibration step (Section **??**)
`b_inc`	Use `support`: the width of the range of the increment
`b_0`	Use `support_0`: the width of the range of the initial damage
`b_new`	Use `support_new`: the width of the range of the damage after a replacement
`n_workers`, `warm_start`	Remove them. Parallel runs and warm starts between horizons are options of the sweep functions (Sections **??** and **??**)
`depot_capacity`	Remove it. The availability of the depot is not limited at fleet level (Section **??**)
`C_P`	Remove it. The loop constraint is a hard constraint and has no penalty (Section **??**)
`C_S`	Use `C_D`, with the same meaning
`alpha`	Use `tau`, with the same meaning
`xi`, `repair_rho`	Use `rho`, with the same meaning
`mu_new`	Use `replacement_mu`, with the same meaning
`v_new`	Use `replacement_v`, with the same meaning

Consistency Checks

The solver must validate all of the following before it builds the model. A violation raises a `ValueError`. Its message names the key and, for a cell value, the cell (i, ℓ). A check of one model runs only at the cells of that model, on the entries where the key applies. Each item gives the rule and, after the dash, the reason for it.

Keys, shapes and entries.

- *Keys.* Every key is in the tables of Section 1. The keys `F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep` and `C_D` are present, and so are the keys that each cell type requires (Section 3). — A misspelled or missing key must not change the problem without a message.
- *Shapes.* Each value has one of the shapes of its table row. The last axis of an operating profile has H_2 entries, and that of a transitory profile has H_1 entries. `component_names` has L entries. — Silent broadcasting by the array library could read a value for another cell.
- *Entries.* At a cell where a key applies, every entry is a finite number, or a string for a selector; `null` is an error. — A missing value at a cell must not take a default silently.

Sizes and selectors.

- F, M and L are positive integers, and $F > M$. — Every mission needs a train on every day; with $F = M$ no train could be in the depot, and no repair could take place.

- H is an integer ≥ 1, or a pair of integers $H_1, H_2 \geq 1$. — Each phase has at least one day.
- `model` entries are `gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`. `repair_model` entries are `ard1` or `ardinf`, for every model. — Another value names no model or no repair law.
- `bound_method` entries at rainflow cells are `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`. — A rainflow cell without one of these bounds has no reliability row.
- `reliability_impl` entries are `exact`, `tangent` or `pwl` at every cell; at a cell with a linear row (IG, Ga, Mk, Ch) a valid value is ignored, and the form `exact` is built. `encoding` entries are `bigm` or `indicator`. — An unknown value must raise an error at every cell; reading it as `exact` would turn a linear model into a nonconvex one without a message.
- `penalty_type` is `inf_norm` or `quadratic`. If $C_D > 0$ and `penalty_type` is `quadratic`, every cell is built in the form `exact`; a cell with a linear row always is. — The quadratic penalty is a quadratic row, and the linear forms exist to give a linear model.

Reliability, repair and initial state.

- $\tau_{i\ell} > 0$, and finite. — The threshold is a positive degradation level.
- $\varepsilon_{i\ell} \in (0, 1)$, and $\varepsilon_{i\ell} < 0.5$ at G and W cells. — A probability lies in $(0, 1)$; the squared normal quantile of the Gaussian and Wiener rows is positive only below 0.5 (Section 1.3).
- $\rho_{i\ell} \in (0, 1]$. — A repair removes a positive fraction of the level, at most all of it.
- $0 \leq \mu_{0,i\ell} < \tau_{i\ell}$. — A component at its threshold has failed already.
- v_0, `replacement_mu`, `replacement_v`, `support_0`, `support_new`, R_0 and R^{new} are ≥ 0. — They describe a damage, a variance, a width or a squared width.
- `tangent_ref` $\in [0, 1)$ at cells with form `tangent`, and `pwl_points` is an integer ≥ 1. — At `tangent_ref` $= 1$ the tangent row reduces to $Q \leq 0$, and a value outside $[0, 1]$ has no tangent point in $[0, \tau]$.
- Starting anchors: $0 \leq$ `anchor_mu_0` $\leq \mu_0$, $0 \leq$ `anchor_v_0` $\leq v_0$, $0 \leq$ `anchor_R_0` $\leq R_0$ and $0 \leq$ `anchor_K_0` $\leq K_0$, at the cells that read them. — Damage only grows between interventions, so the level after the last intervention cannot exceed the initial level.
- If `replacement_as_new` is `true`, then `replacement_mu`, `replacement_v`, `support_new`, `R_new` and `cgf_new` are 0 at every cell that reads them. The check does not run when the flag is `false`. — The shortcut means a new part, and a non-zero value would be discarded without a message.

Natural ranges and replacement states.

- Every initial value (μ_0, v_0, R_0, K_0) and every replacement value (μ^{new}, v^{new}, R^{new}, K^{new}, checked only if `allow_replacement` is `true`) lies in $[0, M_s]$, where M_s is the natural range of its state: τ for a mean; $V_{\max} = \tau^2/\big(\Phi^{-1}(1-\varepsilon)\big)^2$ for the variance at G and W cells; $\varepsilon\tau^2/(1-\varepsilon)$ for the variance at Ca cells; $\tau(\tau - 2c)/(2\kappa)$ at Be cells, with $c = \kappa b/3$ and $\kappa = \ln(1/\varepsilon)$; $2\tau^2/\kappa$ for R at Ho cells; $s\tau - \kappa$ for K at Ch cells (Sections **??** to **??**). A starting anchor lies between 0 and the initial value of its state, so it lies in the range as well. — The big-M rows take the natural range as their constant, so a value outside it violates a row at once.
- If `allow_replacement` is `true`, the replacement state of every cell satisfies the reliability row of the cell, with $(\mu^{\text{new}}, v^{\text{new}}, R^{\text{new}}, K^{\text{new}})$ in place of the tracked states (Sections **??** to **??**). — The row holds on the day of a replacement; otherwise a replacement is never feasible.

Increments.

- $0 \le \mu^{\mathrm{inc}}_{i\ell jk} < \tau_{i\ell}$ for every mission j and every day $k \in \{1, \dots, T\}$. — A mission with mean τ or more fails the component in one day; the mean 0 is allowed, so that a mission that does not wear a component can be written as 0.
- Where $\mu^{\mathrm{inc}}_{i\ell jk} = 0$, the values $v^{\mathrm{inc}}_{i\ell jk}$, $b^{\mathrm{inc}}_{i\ell jk}$ and $K^{\mathrm{inc}}_{i\ell jk}$ are 0. — A non-negative increment with mean 0 is 0 almost surely, so it has no variance, no range and a zero cumulant generating function.
- The values $v^{\mathrm{inc}}_{i\ell jk}$, $b^{\mathrm{inc}}_{i\ell jk}$ and $K^{\mathrm{inc}}_{i\ell jk}$ are ≥ 0. A transitory profile has its operating profile. — A variance and a width are not negative, and the lookup (1) needs the operating entries after the transitory phase.

Gaussian cells.

- $\mu_0 \ge 3\sqrt{v_0}$ and $\mu^{\mathrm{inc}}_{i\ell jk} \ge 3\sqrt{v^{\mathrm{inc}}_{i\ell jk}}$ for every mission j and day k. — A Gaussian increment is then negative with probability at most about 0.13 %, so the damage stays non-negative in practice.

Inverse Gaussian cells.

- $\eta_{i\ell} > 0$. — The dispersion of the inverse Gaussian law must be positive.

Wiener cells.

- $\sigma_{i\ell} > 0$ and $v_{0,i\ell} \ge 0$. — The diffusion constant is positive; a known initial damage has variance 0 and is a valid input.

Gamma cells.

- Every entry of `gamma_beta` and of `gamma_beta_trans` is finite and > 0. — The rate of a Gamma law is positive, and the smallest rate sets the cap (Section **??**).
- `gamma_beta_0` and `gamma_beta_new`, where they are given, are finite and > 0. — A rate is positive; an omitted rate means a known damage, which has no Gamma law and no rate.

Cells with a variance cap.

- If `v_max_user` is given at a G, W or RF (Ca, Be) cell, then $V^{\mathrm{user}}_{\max,i\ell} > 0$ and $V^{\mathrm{user}}_{\max,i\ell} \ge v_{0,i\ell}$. — A cap below the initial variance makes the problem infeasible.

Rainflow cells.

- Bernstein: $v^{\mathrm{inc}}_{i\ell jk} \le (b^{\mathrm{inc}}_{i\ell jk})^2/4$ for every mission j and day k, $v_0 \le (b^0)^2/4$ and $v^{\mathrm{new}} \le (b^{\mathrm{new}})^2/4$. — Popoviciu's inequality (below): a law on a range of width w has variance at most $w^2/4$, so a larger variance contradicts the width.
- Bernstein: $\tau_{i\ell} > 2\kappa\, b_{i\ell}/3$, with $\kappa = \ln(1/\varepsilon_{i\ell})$ and $b_{i\ell}$ the support constant of Section **??**. — Otherwise the certificate admits no positive variance: it is infeasible at zero damage, and at equality only $\mu = v = 0$ survives, so that the big-M rows break.
- Bernstein with the form `tangent`: `tangent_ref`$\cdot\tau_{i\ell} < \tau_{i\ell} - c_{i\ell}$, with $c_{i\ell} = \kappa\, b_{i\ell}/3$. — Otherwise no point with $v \ge 0$ satisfies the tangent row, so the row is empty (Section **??**).
- Chernoff: `s_chernoff` is given and $s_{i\ell} > \ln(1/\varepsilon_{i\ell})/\tau_{i\ell}$. — The quantity K is not negative, so the row $K - s\tau \le \ln \varepsilon$ needs $s\tau \ge \ln(1/\varepsilon)$; with a smaller s every schedule is infeasible, and at equality only a component without damage is feasible.
- Chernoff: $K^{\mathrm{inc}}_{i\ell jk} \ge s_{i\ell}\, \mu^{\mathrm{inc}}_{i\ell jk}$ for every mission j and day k, $K_0 \ge s\,\mu_0$ and $K^{\mathrm{new}} \ge s\,\mu^{\mathrm{new}}$. — Jensen's inequality: the cumulant generating function of a random variable at $s > 0$ is at least s times its mean.

Run options.

- `objective_mode` is `total`, `operating_average` or `evaluation_total`. In the last mode, `evaluation_horizon` is an integer $E > H_1$. — The projected cost needs at least one operating day beyond the transitory phase.
- `mip_gap` ≥ 0; `time_limit`, `progress_interval_seconds` and `relaxation_time_limit` are > 0; `gurobi_params` is a mapping; `fast`, `relaxation_warm_start`, `allow_replacement` and `replacement_as_new` are flags. — A value outside its range must raise an error and must not be changed silently.

Remark (Popoviciu's inequality). Let X take values in an interval of width w, with midpoint c. Then $\operatorname{Var} X \leq \mathbb{E}\,(X - c)^2 \leq (w/2)^2$. The width is the only information on the range, and the range may start at any point $a \geq 0$. The inequality $\operatorname{Var} X \leq w^2/4$ holds for every such position, so it is a necessary condition for the pair (variance, width). For example, increments in an interval of width 0.2 have variance at most 0.01; a two-point law on $\{0, 0.2\}$ with probability $1/2$ each has mean 0.1 and reaches 0.01. A condition of the form $v \leq \mu\,(b - \mu)$ assumes that the range starts at 0, so it is not necessary for a width.

Output Specification

The function `solve()` returns one result dictionary. If its argument `results_path` is not `None`, the function also writes the same content to a file. Every call solves one problem, with the horizon H of the input; a solve for several horizons calls `solve()` once per horizon (Section **??**).

Result Dictionary

The array values are `numpy.ndarray` objects. The result holds no Gurobi object. In the column *Shape*, a dash marks a scalar or a string.

Key	Shape	Description
Status and quality		
`status`	—	Status of the solve: `optimal`, `time_limit`, `suboptimal`, `interrupted`, `solution_limit`, `node_limit`, `iteration_limit`, `infeasible`, `inf_or_unbounded`, `unbounded`, or `gurobi_status_`n for any other Gurobi status code n
`objective`	— or `null`	Objective value of the best solution found; `null` if there is none
`mip_gap`	— or `null`	Relative MIP gap of the best solution at the end of the solve; `null` if there is no solution. The input key `mip_gap` is the gap that was requested
`bound`	— or `null`	Best bound of Gurobi, if it is available
Problem		
`F`, `M`, `L`	—	Numbers of trains, missions and components
`H`	integer or $[H_1, H_2]$	Horizon as written in the input
`H1`, `H2`, `T`	—	Lengths H_1 and H_2 of the two phases, and $T = H_1 + H_2$
`model`	$F \times L$ list of strings	Degradation model of each cell, as in the input
`component_names`	list of L strings	Names of the components
`tau`, `mu_0`, `v_0`	$F \times L$	Threshold, initial mean and initial variance of each cell, after broadcasting

Key	Shape	Description
`bound_method`, `repair_model`, `reliability_impl`, `encoding`	string, or $F \times L$ list of strings	The selector of each cell after broadcasting: one string if all cells agree, else a list. `bound_method` is `null` at a cell that is not rainflow
`objective_mode`, `evaluation_horizon`	—	The objective mode, and E if it was given
Solution (`null` if no solution was found)		
`x`	$F \times (M{+}1) \times T$	Assignment: $x_{ijk} = 1$ if train i flies mission $j \geq 1$ on day k, and $x_{i0k} = 1$ if it is in the depot. A train with $x_{ijk} = 0$ for every $j \in \{0, \ldots, M\}$ is idle
`m`	$F \times L \times T$	Repair: $x^m_{i\ell k}$
`r`	$F \times L \times T$	Replacement: $x^r_{i\ell k}$; zero if `allow_replacement` is `false`
`idle`	$F \times L \times T$	$\text{idle}_{i\ell k} = x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k}$: component ℓ is in the depot without an intervention
`mu`	$F \times L \times T$	Tracked mean degradation
`v`	$F \times L \times T$	Tracked variance. `NaN` in memory and `null` in a file at cells that track no variance (IG, Ga, RF with Mk, Ho or Ch)
`z`	$F \times L \times T$	Repair-cost variable. `NaN` in memory and `null` in a file at cells with $C_{R,i\ell} = 0$, where the variable is not built. On the transitory days of the mode `operating_average` the value is computed after the solve (below)
`R`, `K`	$F \times L \times T$	Tracked Hoeffding budget R and Chernoff quantity K. `NaN` in memory and `null` in a file at the cells that do not track the quantity: R is tracked at RF (Ho) cells, K at RF (Ch) cells
`g_mu`, `g_v`, `g_R`, `g_K`	$F \times L \times T$	Tracked anchors $g^\mu_{i\ell k}$, $g^v_{i\ell k}$, $g^R_{i\ell k}$ and $g^K_{i\ell k}$: the value of the state after the last intervention up to day k. `NaN` in memory and `null` in a file at the cells that do not track the anchor: only cells with `repair_model ard1` track one, `g_mu` for every model, `g_v` at G and RF (Ca, Be) cells, `g_R` at RF (Ho) cells, `g_K` at RF (Ch) cells
`u`	—	Damage regularisation value (below)
Cost report (`null` if no solution was found)		
`step_costs`	T	Cost c_k of each day, Equation (4)
`J_initialization`, `J_op`, `J_op_average`	—	J_{init}, J_{op} and J_{op}/H_2, Equation (5)
`peak_damage`, `damage_penalty`	—	u and $C_D\, u$
`operating_objective`, `J_total`	—	$J_{\text{op}}/H_2 + C_D\, u$ and $J_{\text{init}} + J_{\text{op}} + C_D\, u$
`projected_evaluation_cost`, `evaluation_objective`	— or `null`	$J_{\text{init}} + (E - H_1)\, J_{\text{op}}/H_2$ and the same plus $C_D\, u$; `null` if `evaluation_horizon` is not given
`component_costs`	mapping	Cost coefficients used: `C_M` and `C_D` (scalars), `C_R` and `C_rep` ($F \times L$)
Reports		
`performance`	mapping	Sizes and times of the solve (Section 2.3)
`optimization_progress`	list of mappings	Progress of the incumbent and the bound (Section 2.3)

Key	Shape	Description
`warm_start`, `relaxation_warm_start`	mappings	What each warm start did (Section 2.3)

Rules for the Values

Solutions and status. The solution arrays are present whenever Gurobi has found at least one solution, and then `status` tells how the solve ended. For example, a solution found before the time limit is reported with the status `time_limit`. If no solution was found, the solution arrays and the cost report are `null`, and `objective` and `mip_gap` are `null`. The four reports of the last block of the table are present in both cases.

Tracked values. The arrays `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R` and `g_K` hold the values of the tracked variables of the model, and `z` holds the repair-cost variable. The result carries every tracked state and anchor, so that a validator can check every loop row of Section **??**. The dynamics rows are one-sided (Section **??**), so these values, and the costs computed from them, are upper bounds, not exact values.

Repair cost on transitory days. In the mode `operating_average`, the variable z is not built on the days of the transitory phase, because their cost does not enter the objective (Section **??**). For these days, at a cell with $C_{R,i\ell} > 0$, the result holds the value computed after the solve from the arrays `mu` and `g_mu`:

$$z_{i\ell k} = \begin{cases} \rho_{i\ell}\left(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1}\right) & \text{if } x^{m}_{i\ell k} = 1, \\ 0 & \text{otherwise,} \end{cases} \qquad k = 1, \dots, H_1, \tag{2}$$

with $\mu_{i\ell 0} = \mu_{0,i\ell}$, $g^{\mu}_{i\ell 0}$ the starting anchor, and $g^{\mu} \equiv 0$ under `ardinf`. It is the amount that the repair removes, in mean units, computed on the tracked values. The cost report uses these values.

Damage regularisation. The value u is recomputed from the arrays `mu`,

$$u = \max_{i,k} \sum_{\ell=1}^{L} \mu_{i\ell k} \quad (\texttt{inf_norm}), \qquad u = \max_{i,k} \sum_{\ell=1}^{L} \mu^{2}_{i\ell k} \quad (\texttt{quadratic}). \tag{3}$$

It is reported also when $C_D = 0$, where the variable u is not built.

Depot, idle and intervention. A row of zeros $x_{i1k} = \cdots = x_{iMk} = x_{i0k} = 0$ is an idle day of train i: no mission and no depot. On a depot day ($x_{i0k} = 1$) each component is repaired, replaced or in the depot without an intervention; the last case is `idle`. Hence $x^{m}_{i\ell k} + x^{r}_{i\ell k} + \text{idle}_{i\ell k} = x_{i0k}$.

Cost report. The cost of day k and the phase costs are

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F}\sum_{\ell=1}^{L} C_{R,i\ell}\, z_{i\ell k} + \sum_{i=1}^{F}\sum_{\ell=1}^{L} C_{\text{rep},i\ell}\, x^{r}_{i\ell k}, \tag{4}$$

$$J_{\text{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\text{op}} = \sum_{k=H_1+1}^{T} c_k. \tag{5}$$

The value `objective` is the value of the model objective. Up to the tolerance of the solver, it equals `J_total` if `objective_mode` is `total`, `operating_objective` if it is `operating_average`, and `evaluation_objective` if it is `evaluation_total`.

Reports

`performance`. The mapping has these entries; an entry that Gurobi does not give is `null`.

Group	Entries
Size of the model	`gurobi_version`, `variables`, `continuous_variables`, `integer_variables`, `binary_variables`, `linear_constraints`, `general_constraints`, `indicator_constraints`, `quadratic_constraints`, `nonzeros`
Work of Gurobi	`solutions_found`, `gurobi_runtime_seconds`, `branch_and_bound_nodes`, `simplex_iterations`, `barrier_iterations`, `work_units`
Wall-clock times, in seconds	`model_construction_seconds`, `optimizer_call_seconds`, `solution_extraction_seconds`, `backend_wall_seconds`
Settings	`requested_time_limit_seconds`, `requested_mip_gap`, `threads`, `seed`

`optimization_progress`. A list of records with the entries `runtime_seconds`, `incumbent`, `best_bound`, `relative_gap`, `nodes` and `solutions`. The incumbent, the bound and the gap are `null` while Gurobi has none. A record is made at the first progress report of Gurobi, then at least `progress_interval_seconds` after the previous record, and whenever the number of solutions changes; the last record is the final state of the solve.

`warm_start`. A mapping with the entries `applied` (`true` when at least one value was set), `values` (the number of values set), `mode` (`"start"`, `"hint"` or `"both"`, and `null` without a source), `source_horizon` (the pair H_1, H_2 of the source, or `null`) and `completion`. The entry `completion` is a mapping with `requested`, `status`, `fixed_integer_variables`, `values_copied`, `runtime_seconds` and `iis_constraints`. The value of `status` is `"not_requested"`, `"completed"`, `"infeasible"`, `"infeasible_or_unbounded"`, `"time_limit"`, or `"status_`q`"` with the Gurobi status code q. The entry `iis_constraints` is a list of at most 50 names of rows of an irreducible infeasible subset; it is empty unless the status is `"infeasible"`. Section **??** defines the mechanisms.

`relaxation_warm_start`. A mapping with the entries `enabled`; `status` (`"disabled"` when the option is false, `"skipped"` when it is true and a start or a hint was applied, otherwise the Gurobi status of the relaxed copy as a string); `relaxation_objective`; `values_copied`; and `runtime_seconds`.

Result Files

The function `solve` writes the result to the file `results_path`.

- The default value of `results_path` is `"output.yaml"`. The value `None` writes no file and only returns the dictionary.
- A path without extension gets the extension `.yaml`. The extensions `.yaml` and `.yml` give YAML, `.json` gives JSON, and `.h5` and `.hdf5` give HDF5. A path with any other extension raises a `ValueError`.
- The extension, and the directory of the path, are checked before the model is solved, so that a long solve is not lost. The directory must exist and be writable.
- YAML and JSON files hold plain data: lists, numbers, strings and `null`. `NaN` is written as `null`. The files load with `yaml.safe_load` and `json.load`.
- A key whose value is `None` is not written, except `objective`, which is written as `null`.
- The layout of an HDF5 file is that of Section 3.2.

Input File Schema

Format Rules

The supported formats are **YAML** (`.yaml`, `.yml`), **JSON** (`.json`) and **HDF5** (`.h5`, `.hdf5`). The format follows the extension of the path, in lower case. Any other extension raises an error. One reader serves every function that takes an input path.

- A file holds one mapping from the keys of Section 1 to values. The three formats hold the same keys with the same meaning.
- An array is a nested list in YAML and JSON. The outermost list runs over the first axis of the shape, in the order of Section 1: train, component, mission, day.
- A key that some cell reads is required if it has no default. A key that no cell reads may be omitted. For a key that only some cells read, the entries of the other cells are `null`.
- YAML is read in the safe mode: a file cannot create program objects.

HDF5 Layout

An HDF5 file has a flat layout: every key is stored at the root of the file, as an attribute or as a dataset. The reader makes one entry of every root attribute and of every root dataset, so every key of Section 1 can be read from HDF5. A byte string is decoded as UTF-8. A text attribute that begins with { or `[` is parsed as JSON.

Input file.

- A scalar number, a flag or a single string is an attribute, or a dataset of shape ().
- `H` is an integer, or two integers, as an attribute or as a dataset.
- An array of numbers is a dataset of type `float64` with the shape of the key. A `null` entry is `NaN`.
- A selector array and `component_names` are datasets of strings, or JSON text. A `null` entry of a selector is the empty string.
- `gurobi_params` is JSON text.

Output file. The result keys of Section 2 are stored by this rule: a number or a string is an attribute; an array of numbers is a dataset of type `float64`; a list of strings, a mapping or a list of records is JSON text in an attribute. A key whose value is absent is not stored. The table below lists the placement of every key. The arrays `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K` and `z` hold `NaN` where the quantity is not tracked, because a `float64` dataset has no `null`.

Placement	Keys
Attribute: number or string	`status`, `objective`, `mip_gap`, `bound`, `F`, `M`, `L`, `H` (one or two integers), `H1`, `H2`, `T`, `objective_mode`, `evaluation_horizon`, `u`, `J_initialization`, `J_op`, `J_op_average`, `peak_damage`, `damage_penalty`, `operating_objective`, `J_total`, `projected_evaluation_cost`, `evaluation_objective`; the selectors `bound_method`, `repair_model`, `reliability_impl` and `encoding` when they are one string
Dataset (`float64`)	`x`, `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K`, `z`, `m`, `r`, `idle`, `step_costs`, `tau`, `mu_0`, `v_0`

Placement	Keys
JSON text in an attribute	`model`, `component_names`, `component_costs`, `performance`, `optimization_progress`, `warm_start`, `relaxation_warm_start`; the selectors `bound_method`, `repair_model`, `reliability_impl` and `encoding` when they differ between cells

Keys by Cell Type

The table below lists, for each cell type, the keys that the cell requires and the optional keys that it reads in addition to those of the first row. A transitory profile (`*_trans`) is optional for every profile key. The optional keys of the first row are read at every cell; the others only at cells of the type.

Cell type	Required	Optional (default in Section 1)
Every cell	`F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep`, `C_D`	`L`, `repair_model`, `reliability_impl`, `encoding`, `tangent_ref`, `pwl_points`, `anchor_mu_0`, `replacement_mu`, `replacement_as_new`, `allow_replacement`, `penalty_type`, `component_names`, `mu_trans`, the run options
G	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
IG	`eta`	none
W	`sigma`	`v_0`, `replacement_v`, `v_max_user`
Ga	`gamma_beta`	`gamma_beta_0`, `gamma_beta_new`, `gamma_beta_trans`
RF, Mk	`bound_method` with the value `markov`	none
RF, Ca	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
RF, Ho	`support`	`R_0`, `R_new`, `anchor_R_0`, `support_trans`
RF, Be	`v`, `support`	`v_0`, `support_0`, `replacement_v`, `support_new`, `anchor_v_0`, `v_max_user`, `v_trans`, `support_trans`
RF, Ch	`cgf`, `s_chernoff`	`cgf_0`, `cgf_new`, `anchor_K_0`, `cgf_trans`

Every rainflow cell also needs the key `bound_method` when its bound is not Cantelli. `anchor_mu_0` and the other starting anchors are read only at cells with `repair_model` equal to `ard1`.

YAML Example (Mixed Models)

The example has $F = 3$ trains, $M = 2$ missions and $L = 2$ components. The six cells use Gaussian, inverse Gaussian, Wiener, Gamma, rainflow with Bernstein, and rainflow with Chernoff. It uses an array, a list and a scalar for cell values, a profile of shape (L, M) and a transitory profile of shape (L, M, H_1). It passes every check of Section 1.12.

```
F: 3
M: 2
L: 2
H: [2, 3]                # H1 = 2, H2 = 3, so T = 5
component_names: [Wheelset, Brake pad]

model:
  - [gaussian, inverse_gaussian]
```

```
  - [wiener,   gamma]
  - [rainflow, rainflow]
repair_model:
  - [ard1,   ard1]
  - [ardinf, ard1]      # the default is ardinf
  - [ard1,   ard1]
bound_method:           # read at rainflow cells only
  - [null,      null]
  - [null,      null]
  - [bernstein, chernoff]
reliability_impl:       # linear-row cells (IG, Gamma, Chernoff): null
  - [tangent, null]
  - [pwl,     null]
  - [exact,   null]
encoding:
  - [bigm, bigm]
  - [bigm, bigm]
  - [bigm, indicator]

tau: 1.0
epsilon:
  - [0.05, 0.05]
  - [0.05, 0.05]
  - [0.10, 0.10]
tangent_ref: 0.4        # read at the tangent cell only
pwl_points: 6           # read at the pwl cell only
rho: [0.8, 0.7]         # per component, the same for every train

mu_0:
  - [0.12, 0.10]
  - [0.15, 0.08]
  - [0.09, 0.14]
v_0:
  - [0.0004, null]
  - [0.0005, null]
  - [0.0004, null]
support_0:              # Bernstein: width of the initial range
  - [null, null]
  - [null, null]
  - [0.05, null]
anchor_mu_0:            # ard1 cells only; at most mu_0
  - [0.10, 0.06]
  - [null, 0.05]
  - [0.00, 0.10]
anchor_v_0:             # ard1 cells with a variance state
  - [0.0002, null]
  - [null,   null]
  - [0.0002, null]
anchor_K_0:             # ard1 cell with Chernoff; K_0 = s * mu_0 = 0.70
  - [null, null]
  - [null, null]
  - [null, 0.50]

replacement_mu: 0.02
replacement_v:
  - [0.00004, null]
  - [0.00004, null]
  - [0.00004, null]
```

```
support_new:
  - [null, null]
  - [null, null]
  - [0.03, null]
replacement_as_new: false
allow_replacement: true

# profiles, axes (L, M): one row per component, one column per mission
mu:
  - [0.06, 0.05]
  - [0.04, 0.07]
mu_trans:               # axes (L, M, H1): days 1 and 2
  - [[0.07, 0.06], [0.06, 0.05]]
  - [[0.04, 0.04], [0.07, 0.07]]
v:
  - [0.0001, 0.0001]
  - [null,   null]
support:
  - [0.05, 0.04]
  - [null, null]
cgf:                    # at s_chernoff = 5; at least s * mu
  - [null, null]
  - [0.22, 0.38]
s_chernoff:
  - [null, null]
  - [null, null]
  - [null, 5.0]

sigma:
  - [null, null]
  - [0.03, null]
  - [null, null]
eta:
  - [null, 0.15]
  - [null, null]
  - [null, null]
v_max_user:
  - [null, null]
  - [0.05, null]
  - [null, null]
gamma_beta:
  - [null, null]
  - [null, 20.0]
  - [null, null]
gamma_beta_new:         # gamma_beta_0 is omitted: the initial damage is
    known
  - [null, null]
  - [null, 20.0]
  - [null, null]

C_M: 1.0
C_R: [2.0, 1.5]
C_rep: [10.0, 8.0]
C_D: 1.5
penalty_type: inf_norm
objective_mode: evaluation_total
evaluation_horizon: 30
```

```
verbose: 0
mip_gap: 0.05
time_limit: 600
fast: false
gurobi_params: {Threads: 4}
progress_interval_seconds: 10
relaxation_warm_start: false
```

In the example, the inverse Gaussian cell uses the law `ard1` with the starting anchor $0.06 \le \mu_0 = 0.10$. The Gamma cell has the starting anchor $0.05 \le \mu_0 = 0.08$ and no `gamma_beta_0`, so that its initial damage is known. The Chernoff cell has $K_0 = 5 \cdot 0.14 = 0.70$ by default, so that its starting anchor 0.50 is allowed. The Bernstein cell has the support constant $b = \max(0.05, 0.03, 0.05) = 0.05$, so that $2\kappa b/3 = 0.077 < \tau = 1$ with $\kappa = \ln 10$.

References

References

[1] L. Doyen and O. Gaudoin. Classes of imperfect repair models based on reduction of failure intensity or virtual age. *Reliability Engineering & System Safety*, 84(1):45–56, 2004. doi:10.1016/S0951-8320(03)00173-X. Read in the authors' preprint submitted on 25 October 2002, whose page numbers are cited.

[2] S. Mercier and I. T. Castro. Stochastic comparisons of imperfect maintenance models for a gamma deteriorating system. *European Journal of Operational Research*, 273(1):237–248, 2019. doi:10.1016/j.ejor.2018.06.020.

[3] F. Roosta-Khorasani and G. J. Székely. Schur properties of convolutions of gamma random variables. *Metrika*, 78:997–1014, 2015. doi:10.1007/s00184-015-0537-9. Read in arXiv:1601.04731v1, whose page numbers are cited.

[4] G. J. Székely and N. K. Bakirov. Extremal probabilities for Gaussian quadratic forms. *Probability Theory and Related Fields*, 126:184–202, 2003. doi:10.1007/s00440-003-0262-6.